

Seminar 3

Object-Oriented Design, IV1350 Seminar 3

Lucas Burgos, lburgos@kth.se

3 maj 2026

1. Introduction

This report presents the code of a program for the Repair Electric Bike scenario in the course Object oriented design IV1350. The purpose of this report is to showcase my solution of Seminar 3, which should demonstrate the requirements of high cohesion, low coupling and high encapsulation. This report focuses on the implementation of the program rather than the design. The solution will besides code also show an example printout in the terminal, and five unit test classes for covering the most important logic in the controller, model and integration layers. This report includes a description of my method, a description of my resulting code and finally a discussion to adress some thoughts and evaluations of the implementation.

2. Method

The implementation of my design from seminar 2 was made using the language java, following the same MVC structure used in seminar 2. I began by creating the file structure of the program by creating the packages (folders) View, Controller, Model, Integration and Startup, and then creating and naming the files of which is below the folders. This made my work of writing code easier.

After creating the file structure, I began from the bottom and worked my way up. First, creating the CustomerRegistry and RepairOrderRegistry. Since the array of all the customers should remain even if an repairOrder is not accepted, the list is declared as final. Its also private since only the CustomerRegistry should know all the customers, using separate methods to look up and return only specific customers to other sections of the program. I then proceeded to add two example customers hard coded into the CustomerRegistry for later testing, and then creating the method findCustomerByPhoneNumber. If a matching phone number is found, the customer is returned. If not, nothing is returned (null). In the same manner, I built the RepairOrderRegistry. First creating the private and final ArrayList and later the methods to generate a repairOrderId, adding a repairOrder object to the array and to find a repairOrder in the registry. While creating these, I worked on the model in parallel to make the coding of the two registers easier such as creating the Customer, repairOrder and Bike objects.

When the integration and model layers had a good skeleton, I started working on the Controller. The controller coordinates calls between view, integration and model. Much like

building the registries, I worked on some of the model objects and the printer in parallel of building the controller. First, I created the class and gave it four private objects, where all but customer are final. Then, I created a public controller which contains the CustomerRegistry, RepairOrderRegistry and Printer. I then link the objects to the classes final objects. Thereafter, I create the relevant operations; findCustomer, createRepairOrder, repairorderId, findrepairOrder, addDiagnosticReport, and so on. This took a while to finalise. I also added the missing objects in the model like DiagnosticReport, RepairOrderState and RepairTask.

When the skeleton of the Controller were implemented, I started coding the view. The View stores a reference to the Controller and uses it to call the system operations. This keeps the simulated user interface separated from most of the application logic. Thereafter, I implemented some hard coded sample-code in the view to satisfy the assessment criteria and task list for seminar three. The sample code runs through the whole user case step by step, from searching up a customer and its bike from the register, creating a repairOrder, creating a list of proposed repair tasks, adding a diagnostic report and estimated completion date, printing the total cost, preparing the order for approval/rejection, and finally accepting the order and printing out the repairOrder receipt. This step by step sample is printed out in the terminal for the user to see.

Finally, I implemented the startup-layer (main) by creating new customerRegistry, repairOrderRegistry, printer, controller and view objects in the class Main. The last row in the class, view.sampleExecution();, initiates the sample code used to display the program functionality in the terminal.

Unit testing

To check that the program functions like intended, I implemented unit tests with JUnit 5 for java. The test classes are placed in a separate folder in my repository, src/test, with the same package structure used in my design from Seminar 2. The tests use the JUnit annotations @Test and @BeforeEach. @Test marks a method as a test method, while @BeforeEach marks a method that is executed before each test. I used @BeforeEach to create fresh objects before every test, which makes the tests independent of each other. To test the programs functionality in different contexts, I decided to use assertions like assertEquals, assertNull and assertEquals to compare the actual result from the tested method with the expected result.

The file ControllerTest checks the functionality of the controller. First I began by creating the class ControllerTest and its private objects controller and printer. Thereafter I began by creating the registries and a printer, while assigning the controller the same registries and a new silent printer. The silent printer is used to store what the printer normally would print out to the terminal, without printing it. The tests can therefore check the output as an object instead of reading the terminal output, as by the name "silent" printer. Then, wrote the code to test the methods in controller. I write specific methods which are responsible for different operations. For example, findCustomerReturnsCorrectCustomerForKnownPhoneNumber() tests if the operation of finding a customer with a phone number works as intended. The same process is done for the Integration layer with CustomerRegistry & RepairOrderRegistry, and for RepairOrder and Amount. RepairOrderTest is testing the central business logic of the EBRS scenario, AmountTest checks calculations and comparisons of amounts. CustomerRegistryTest checks that customers can be found by phone number, and RepairOrderRegistryTest checks that repair orders are stored and found by order id.

I intentionally chose to test core functionality that includes logic, change in states and important rules instead of getters and setters since the task list explicitly stated that other things than getters and setters should be tested.

3. Result

The result is a compilable java program with sample code that implements the basic flow to handle repairOrders for a bike. A user of the program can search for a customer by phone number, create repair orders, add diagnostic reports and add proposed repair tasks, calculate the total cost and handle the scenario of both accepting and rejecting the repair task proposition.

The program is divided into the layers startup, view, controller, model and integration. In startup, the file main creates the programs objects and executes the program. View includes a hardcoded example which calls for the Controller. The controller coordinates the flow between user interface (the terminal for now), the model classes and the integration classes. The model layer includes the programs central objects like RepairOrder, Customer, Bike, RepairTask, DiagnosticReport and Amount. The integration layer simulates as external systems and data storage by creating the array list objects CustomerRegistry, RepairOrderRegistry and Printer.

The unit tests are written with JUnit 5 for the classes that has the most important logic in the program. The tests can be found in a separate folder, src/test, and follows the same layer structure as the rest of the program. The tests covers both normal cases and special ones that can occur. A normal case is creating an order and accepting it when its ready, while a special case could be to try and create an order without a chosen customer associated with it. Both of these, amongst others, can be found in the file named ControllerTest.

To verify that the implemented basic flow can be executed, the program was run through the hard-coded sample execution in View. The sample run shows that the program can find a customer, find the customer's bike, create a repair order, add diagnostic information and repair tasks, calculate the total cost, prepare the order for approval and finally accept and print the repair order.

The resulting files can be found in the submitted url for seminar 3 and down below in references. Please see my previous report for seminar 2 to see my initial design for the program.

Description of the example output in order: (1) The program starts by finding a customer using a phone number. (2) When the customer has been found, the customer's registered electric bike is displayed. (3-4) A new repair order is created with the customer, the bike and the customer's problem description. (5) The program creates two proposed repair tasks with their costs. (6-7) The repair order is updated with a diagnostic report, the proposed repair tasks and an estimated completion date. (7) The total cost is calculated, which in the sample run is $900 + 650 = 1550$. (8) When all required information has been added, the order is marked as ready for customer approval. (9) The accepted repair order is printed in the terminal. (10) The customer accepts the order, which changes its state to ACCEPTED.

Example output (short, full output can be seen in the readme in the GitHub repository):

1. Found customer: Customer{name='Sara Lind', phoneNumber='0701234567', ...}
2. Found bike: Bike{brand='Crescent', model='Elina', serialNumber='SN-12345'}
- 3-4. Created repair order: RepairOrder{orderId=1, state=NEWLY_CREATED, ...}
5. Repair tasks: Replace battery connector, Update motor controller firmware
7. Total cost: 1550
8. Repair order for approval: RepairOrder{orderId=1, state=READY_FOR_APPROVAL, ...}
9. Printer output:
Repair Order
Order ID: 1
State: ACCEPTED
Total Cost: 1550
10. Accepted repair order: RepairOrder{orderId=1, state=ACCEPTED, ...}

4. Discussion

The program was evaluated using the task description, the assessment criteria for seminar 3, the course literature and my design from seminar 2. The discussion below is an evaluation of my implementation.

A clear strength of the implementation is that the program follows the MVC and layer patterns from the design. However, this is not achieved only by placing classes in different packages, but mainly by separating responsibilities between the classes. The startup layer is only responsible for creating the initial objects and starting the sample execution. The view layer simulates user interaction with hard-coded calls. The controller coordinates the system operations and forwards calls to the model and integration layers. The model layer contains the central domain objects and business rules, for example the state changes of a repair order. The integration layer simulates external systems and data storage through registries and the printer.

This separation supports low coupling because the view does not need to know how repair orders are stored, and the model does not need to know anything about the user interface. It also supports high cohesion because each class has a relatively clear responsibility. For example, `RepairOrder` handles the information and rules for one repair order, `RepairOrderRegistry` stores repair orders, `CustomerRegistry` searches for customers and `Printer` handles output.

One limitation in the current implementation is that View does not only communicate with Controller, it also receives and uses model objects such as `Customer`, `RepairOrder`, `RepairTask` and `Amount`. This increases coupling between the view and model layers, since changes in the model classes could also require changes in the view. An alternative design would be to let Controller return DTO objects containing only the information needed by the view. This would keep the model more hidden and make the public interface between view and controller clearer.

The principle of encapsulation has been used widely in the program. In the different classes, their respective attributes has been set to private and can therefore only be accessible by the class in question. An example of this is the file `RepairOrder` and its corresponding class, where status and reparation suggestions are changed within the class, not accessible to the outside. Changes must be made with the help of methods, such as `accept-` or `rejectRepairOrder()`. This makes it easy for the class to also use its own rules. For example, a `repairOrder` cannot be rejected or accepted without there being an diagnostic report, at least one repair task while also being marked as ready for approval. All logic regarding a specific object is handled within itself, and is defined by the class.

The principle of low coupling is something that has become evident while coding the controller. While designing the system in Astah, I did not personally realise the effects of the controller being such a central part of the program. One concern that I mentioned in my report for seminar 2 is that the controller is almost to central according to the pure definition of coupling, while it at the same time should be according to the MVC framework. Leif mentioned a rule of thumb of 4 connections between the controller to other classes. While this is a good rule of thumb, I did not realise in the design stage how huge the controller file would turn out to be. Coding the controller turned out to take much more time than I initially

thought. Some concerns of this is that the controller currently knows of the classes `customerRegistry`, `repairOrderRegistry` and `printer`. One thing to change could be to create public interfaces to minimise the knowledge of other classes. This could be used to later implement a database without changing the controller to much. Another thought could be to minimise the temporal coupling in controller. Some methods has to be called before the other, which could be problematic in the long run. An alternative way could be to take in the nessecary parameters in the beginning, and then let other methods be left. This takes away some of the dependency of certain methods and makes the program easier to maintain and extend .

There are also some thoughts on the tests developed. The tests do not verify the exact output printed to `System.out`. This is a conscious limitation, since the task description states that methods only producing output to `System.out` do not need to be tested. Instead, `ControllerTest` verifies that the accepted repair order is passed to the `Printer`. Another limitation is that empty problem descriptions are currently accepted without validation. In a real system, this could lead to incomplete repair orders.

In summary, the implemented solution of the scenario is adjusted for the tasks given in seminar 3. There is always room for improvement but the core functionality of the program is implemented. The program is object oriented with a design of which thinks of encapsulation, relatively high cohesion and low coupling, even if I have realised that the coupling might not be so low after all. The unit tests strengthens the implemented design by verifying important logic used in the program. The largest limitations of the program for now is that data is hardcoded for the tests and that the user interface is simulated.

Här är en färdig del du kan klistra in i rapporten, exempelvis efter introduktionen eller före metodavsnittet.

****Changes Made for Seminar 5****

After receiving feedback on the Seminar 3 submission, I updated both the implementation and the tests to better follow the MVC principles and the layer structure described in the course literature. The main issue in the previous version was that the view layer had direct access to model objects, and that the controller returned model objects directly to the view. This meant that the view depended on classes such as ``Customer``, ``RepairOrder``, ``RepairTask`` and ``Amount``, which increased coupling between the view and model layers.

To correct this, I introduced a new DTO package: ``se.kth.iv1350.repairebike.dto``. This package contains ``CustomerDTO``, ``BikeDTO``, ``RepairOrderDTO`` and ``RepairTaskDTO``. These classes are used to transfer data between the controller and the view without exposing the internal model objects. The controller still uses the model internally, but its public methods now return DTO objects instead of model objects. For example, methods such as ``findCustomer``, ``createRepairOrder``, ``addDiagnosticReport``, ``prepareRepairOrderForApproval``,

``acceptRepairOrder`` and ``rejectRepairOrder`` now return DTOs.

I also updated the ``View`` class so that it no longer imports or uses any classes from the model layer directly. Instead, it only communicates with the controller and uses DTO objects when printing the hard-coded sample flow. This reduces coupling between the view and model layers and makes the layer responsibilities clearer.

Another issue in the previous version was that several model classes used ``toString()`` to format output for display. The ``RepairOrder`` class also had a method called ``createPrintableStringRepresentation()``, which created formatted output for printing. This mixed presentation logic into the model layer. To correct this, I removed the display-oriented ``toString()`` methods from the model classes and removed ``createPrintableStringRepresentation()`` from ``RepairOrder``. Formatting of output is now handled in the view layer instead.

The ``Printer`` class was also changed. Previously, it received a ``RepairOrder`` model object and formatted it for printing. In the updated version, ``Printer`` only receives already formatted text and prints it to ``System.out``. This keeps the printer focused on output and avoids placing presentation formatting in the model or integration layer.

Finally, I updated the controller tests to match the new design. The tests now verify the controller's DTO-based public interface instead of expecting model objects to be returned. This means the tests now check the corrected MVC structure while still verifying the same important program behavior, such as finding customers, creating repair orders, adding diagnostic reports, calculating total cost and changing order states.

In summary, the updated implementation corrects the three main MVC issues from the previous submission: the view no longer accesses model objects directly, the controller no longer returns model objects to the view, and display formatting has been removed from the model layer. These changes reduce coupling between layers and make the program better aligned with the MVC and layer principles used in the course.

References

Lindbäck, L. (2026). A First Course in Object-Oriented Development: A Hands-On Approach. January 14, 2026 version.

IV1350 Seminar 3 task list

IV1350 Seminar 3 assessment criteria

Seminar 2, my own design in astah

Github url: <https://github.com/lucas17skapar/EBRS-kod.git>

Declaration on the use of AI: I used ChatGPT to speed up my work and the report writing. The AI was used to help structure the code, write comments, remind me of concepts from the course literature and to suggest improvements related to coupling, cohesion, encapsulation, especially when realising that the controller doesn't have so low coupling after all. I verified the AI's output by comparing it with the seminar 3 assignment instructions, the assessment criteria and the course literature.